Les fichiers et les modules

- Les fichiers
- Les modules
  - Formatage des chaînes de caractères
  - Mathématiques
  - Time, datetime
  - Random
  - Graphiques

Coordinatrice : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr
Support du cours: Asma Gabis

P1, P1-BN, P1-P, P1-BDX – 2024/2025

Une partie de ce cours est tirée du cours de Xavier Tannier de Polytech Sorbonne

01

Les fichiers

01

Les fichiers

Pourquoi utiliser un fichier ?

Utilisation des variables dans le programme

```
y = 15

x = 2.8
```

Attribution d'un espace de stockage en mémoire

Mémoire vive

adresse	Espace de stockage
@1	Variable : x Type : float Valeur : 2.8
....	....
.....	....
@2	Variable : y Type : int Valeur : 15

Problèmes:

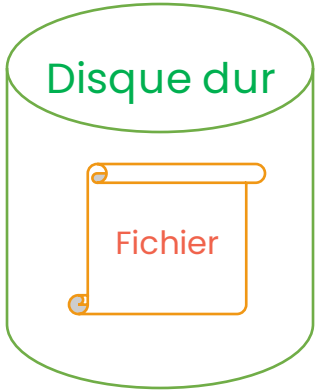
- Si la variable est ré-utilisée, sa valeur initiale est perdue
- Données perdues à la fin du programme

Comment conserver l'information de manière permanente ?

Les fichiers

Pourquoi utiliser un fichier ?

Solution : Stocker les données dans **un fichier** se trouvant sur un support de stockage permanent (ex: disque dur)



Deux types d'interactions avec les fichiers

Lecture : Accéder au contenu du fichier

Écriture : Avoir la possibilité de changer le contenu du fichier (modifier)

Les fichiers

Nommage des fichiers

- Un fichier est stocké en mémoire :
  - Dans un emplacement
  - Sous un nom donné par l'utilisateur

- Le nom d'un fichier accepte tous les caractères exceptés : * $ / * ? < > " | : . espace
- Pour utiliser un fichier dans un programme informatique, il faut donner son nom ainsi que son emplacement. Deux façons possibles:
  - Nommage absolu
  - Nommage relatif

Les fichiers

Nommage des fichiers

Nommage absolu

```
# Chemin Linux, MAC
nom_fichier = '/home/asma/cours_python/fichierTest.txt'



# Chemin Windows : plusieurs façons de le représenter
nom_fichier = 'C:/users/asma/cours_python/fichierTest.txt'
nom_fichier = 'C:\\users\\asma\\cours_python\\fichierTest.txt'
```

Les fichiers

Nommage des fichiers

Nommage relatif : Nommage des fichiers par rapport au répertoire courant

```
# Un fichier dans le répertoire courant :
nom_fichier = 'fichierTest.txt'
# ou
nom_fichier = './fichierTest.txt'

# Un fichier dans le répertoire `cours` contenu dans le répertoire courant
nom_fichier = 'cours/fichierTest.txt'

# Un fichier dans le répertoire parent du répertoire courant
nom_fichier = '../fichierTest.txt'

# Pour reculer de plusieurs répertoires(ici, 3)
nom_fichier = '../../../fichierTest.txt'
```

Le plus recommandé !!

Les fichiers

Ouverture et fermeture d'un fichier en Python

L'instruction open(): Permet d'ouvrir un fichier

Nom_variable = **open(nom_fichier, type d'ouverture)**

Chaîne de caractères : nom absolu ou relatif du fichier à lire.

Exemple :

```
# Ouverture d'un fichier en lecture
f = open("fichierTest.txt", "r")

# Ouverture d'un fichier en écriture
mon_fichier = "./fichierTest.txt"
f = open(mon_fichier, "w")
```

Type ouverture	Effet
r	Pour une ouverture en lecture (READ).
w	Pour une ouverture en écriture (WRITE), à chaque ouverture le contenu du fichier est écrasé. Si le fichier n'existe pas python le crée.
a	Pour une ouverture en mode ajout à la fin du fichier (APPEND). Si le fichier n'existe pas python le crée.

Les fichiers

Ouverture et fermeture d'un fichier en Python

La méthode close(): Tout fichier ouvert doit être fermé après la fin de son utilisation.

Exemple :

```
# Ouverture d'un fichier en lecture
f = open("fichierTest.txt", "r")

# fermeture du fichier ouvert
f.close()
```

Les fichiers

Ouverture et fermeture d'un fichier en Python

Le mot clé with :

- C'est une autre manière d'ouvrir le fichier en Python.
- Permet à la fois d'ouvrir et de fermer un fichier.
- Si pour une raison ou une autre l'ouverture ou la lecture du fichier conduit à une erreur, l'utilisation de **with** garantit la bonne fermeture du fichier.

with open(nom_fichier, type d'ouverture) as nom_variable :
↔ actions

Exemple :

```
# Ouverture d'un fichier en lecture
with open("fichierTest.txt", "r") as f :
    # actions
```

Les fichiers

Lecture d'un fichier en Python

La méthode read():

- Permet de lire l'ensemble du fichier sous forme d'une chaîne de caractère .
- Méthode facile et immédiate
- **MAIS**
  - Le contenu du fichier n'est pas séparé en lignes
  - Le fichier est stocké dans une variable ➔ chargé en mémoire vive ➔ peut être long ou impossible si le fichier est très grand

Exemple :

```
# Ouverture d'un fichier en lecture
with open("fichierTest.txt", "r") as f :
    # Lecture du fichier ouvert
    contenu = f.read()
```

Ici la variable « contenu » est de type str

Les fichiers

Lecture d'un fichier en Python

La méthode readlines() (avec s) :

- Permet de lire l'ensemble du fichier sous forme d'une liste de lignes (de chaînes de caractères).
- Méthode facile et immédiate
- **MAIS**
  - Le fichier est stocké dans une variable ➔ chargé en mémoire vive ➔ peut être long ou impossible si le fichier est très grand

Exemple :

```
# Ouverture d'un fichier en lecture
with open("fichierTest.txt", "r") as f :
    # Lecture du fichier ouvert
    contenu = f.readlines()
for ligne in contenu:
    print(ligne)
```

Ici la variable « contenu » est de type list

Les fichiers

Lecture d'un fichier en Python

Lecture ligne par ligne : Itérations directe sur le fichier

```
# Ouverture d'un fichier en lecture
with open("fichierTest.txt", "r") as f :
    for ligne in f:
        print(ligne)
```

Ici la variable « ligne » est de type str

Les fichiers

Lecture d'un fichier en Python

La méthode readline() (sans s) :

- Permet de lire une seule ligne du fichier sous forme d'une chaîne de caractères.
- À chaque nouvel appel de .readline(), la ligne suivante est renvoyée.
- Associée à une boucle, cette méthode permet de lire un fichier ligne par ligne.

Exemple :

```
# Ouverture d'un fichier en lecture
with open("fichierTest.txt", "r") as f :
    # Lecture du fichier ligne par ligne
    ligne = f.readline()
    while ligne != "":
        print(ligne)
        ligne = f.readline()
```

Ici la variable « ligne » est de type str

Les fichiers

Écriture dans un fichier en Python

La méthode write():

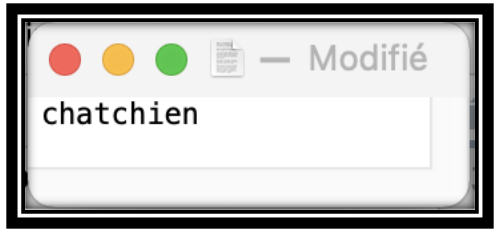
- Permet d'écrire dans un fichier.
- En mode « w », le contenu du fichier est d'abord écrasé avant écriture.
- En mode « a », le contenu écrit s'ajoute après le contenu déjà existant dans le fichier.

Exemple 1 :

```
# Ouverture d'un fichier en écriture
f = open("fichierTest.txt", "w")
# Écriture dans le fichier ouvert
f.write("chat")
f.close()
```

Exemple 2 :

```
# Ouverture d'un fichier en ajout
f = open("fichierTest.txt", "a")
# Écriture dans le fichier ouvert
f.write("chien")
f.close()
```



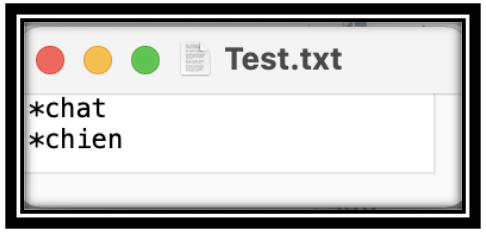
Les fichiers

Ouverture de plusieurs fichiers

Avec l'instruction with, il est possible d'ouvrir deux fichiers (ou plus) simultanément.

Exemple :

```
# Ouverture de deux fichiers: un en lecture, l'autre en écriture
with open("fichierTest.txt", "r") as f1, open("Test.txt", "w") as f2:
    # Lecture du fichier f1
    for ligne in f1:
        # Écriture dans le fichier f2
        f2.write("*" + ligne)
```



Les fichiers

Exercices

- Comment améliorer ce code pour avoir en sortie un mot par ligne ?

```
# Ouverture d'un fichier en écriture
f = open("zoo.txt", "w")
# Écriture dans le fichier ouvert
f.write("chat")
f.write("chien")
f.close()
```

- Comment insérer une information à une ligne se trouvant à un index donné ?
- Quel sera le contenu du fichier « zoo.txt » après l'exécution de ce code ?

```
# Ouverture d'un fichier en écriture
f = open("zoo.txt", "w")
f.close()
# Écriture dans le fichier ouvert
f.write("lion")
```

Les fichiers

Exercices : Solutions

- Ajouter le caractère « \n » à la fin de chaque ligne insérée pour provoquer un retour à la ligne.

```
# Ouverture d'un fichier en écriture
f = open("zoo.txt", "w")
# Écriture dans le fichier ouvert
f.write("chat" + "\n")
f.write("chien" + "\n")
f.close()
```

- Lire le fichier dans une liste (.readlines()) , insérer l'information à l'index souhaité puis écrire le contenu de la liste résultante dans le même fichier en mode « w »
- Aucun contenu. Le programme ne s'exécute pas mais génère une erreur car il y a eu tentative d'écriture sur un fichier déjà fermé.

```
Traceback (most recent call last):
  File "…", line 438, in <module>
    f.write("lion")
ValueError: I/O operation on closed file.
```

02

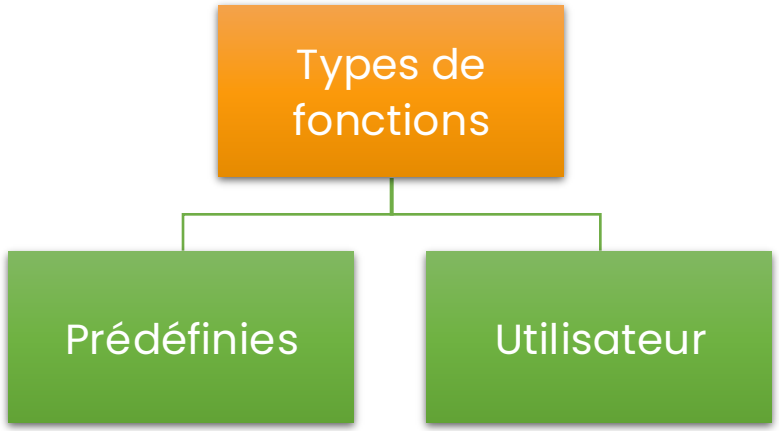
Les modules

02

Les modules

La notion de sous-programme

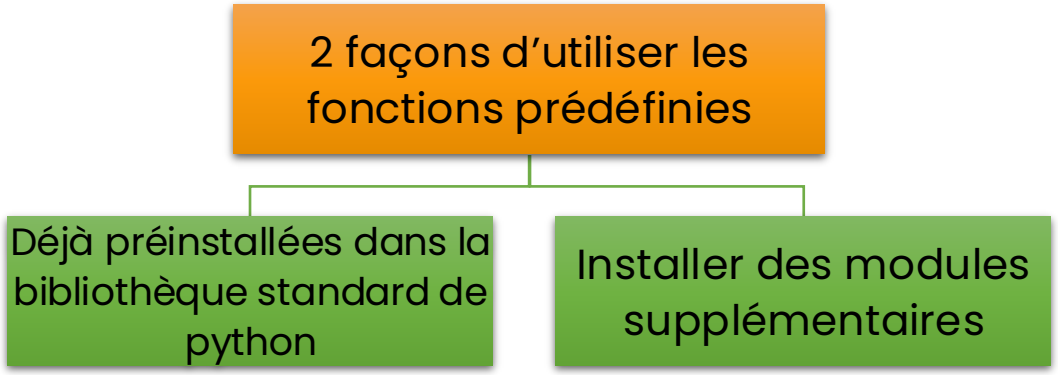
- Un sous-programme est une portion de code isolée dans une fonction si :
  - Une tâche doit être réalisée plusieurs fois par un programme avec des valeurs différentes.
  - La personne qui définit la fonction est différente de celle qui l'utilise.



Les modules

Les fonctions prédéfinies

- Certains traitements sont complexes à effectuer par un algorithme (ex: calcul du sinus d'un angle).
- Tous les langages de programmation ont un certain nombre de fonctions qui permettent de connaître immédiatement ces résultats.
- Plusieurs fonctions prédéfinies ont déjà été utilisées : `print()`, `input()`, `range()`, `len()`



Les modules

Les fonctions préinstallées

Le formatage des données :

Jusque-là, pour composer des chaînes de caractères complexes, nous avons utilisé :

- la fonction **str()** pour convertir les entiers, flottants, etc. en chaînes de caractères.
- l'opérateur + pour concaténer.

Un autre moyen tout aussi simple mais plus puissant : la fonction **format().**

Pour contrôler le format de la chaînes de caractères les caractères « {« et « } » sont utilisés pour créer des trous dans la chaînes de caractères qui peuvent être remplis par :

- Des mots
- Des expressions

Les modules

Les fonctions préinstallées

Le formatage des données :

- Remplacement simple

```
a, b, c = 12, 43.5, 'toto' # a = 12, b = 43,5, c = 'toto'
print('Mes valeurs sont a = {}, b = {} et c = {}'.format(a, b, c))
```

```
Mes valeurs sont a = 12, b = 43.5 et c = toto
```

- Remplacement avec index: les variables dans format() sont indexées à partir de 0. Elles ne sont pas nécessairement données dans l'ordre. Le remplacement se fait par leurs indices.

```
a, b, c = 12, 43.5, 'toto' # a = 12, b = 43,5, c = 'toto'
print('Mes valeurs sont a = {1}, b = {2} et c = {0}'.format(c, a, b))
```

```
Mes valeurs sont a = 12, b = 43.5 et c = toto
```

Les modules

Les fonctions préinstallées

Le formatage des données :

- Remplacement avec nom de variables: la valeur de la variable est affectée dans « format() »

```
print('Mes valeurs sont a = {a}, b = {b} et c = {c}'.format(a=12, b=43.5, c='toto'))
```

```
Mes valeurs sont a = 12, b = 43.5 et c = toto
```

- Remplacement sans instanciation immédiate des paramètres:

```
liste = ['pomme', 'orange', 'kiwi', 'artichaut']
message = "L'élément {} de ma liste est '{}'"
for i in range(len(liste)):
    print(message.format(i, liste[i]))
```

```
L'élément 0 de ma liste est 'pomme'
L'élément 1 de ma liste est 'orange'
L'élément 2 de ma liste est 'kiwi'
L'élément 3 de ma liste est 'artichaut'
```

Les modules

Les fonctions préinstallées

Le formatage des données avec contraintes :

- Alignement :

```
liste = ['pomme', 'orange', 'kiwi', 'artichaut']
for element in liste:
    print('| {:>10} |'.format(element)) # Aligner à droite, sur 10 caractères
```

```
|     pomme |
|    orange |
|      kiwi |
| artichaut |
```

```
liste = ['pomme', 'orange', 'kiwi', 'artichaut']
for element in liste:
    print('{:.>10}'.format(element)) # Aligner à droite, sur 10 caractères, en
complétant avec des .
```

```
.....pomme
....orange
......kiwi
.artichaut
```

Les modules

Les fonctions préinstallées

Le formatage des données avec contraintes :

- Alignement :

```
liste = ['pomme', 'orange', 'kiwi', 'artichaut']
for element in liste:
    print('| {:^10} |'.format(element)) # Centrer sur 10 caractères
```

```
|   pomme    |
|   orange   |
|    kiwi    |
| artichaut  |
```

- Couper une longue chaîne de caractères: avec le « . »

```
liste = ['pomme', 'orange', 'kiwi', 'artichaut']
for element in liste:
    print('| {:.5} |'.format(element)) # afficher au plus 5 caractères
```

```
| pomme |
| orang |
| kiwi  |
| artic |
```

Les modules

Les fonctions préinstallées

Le formatage des données avec contraintes :

- Alignement des nombres : avec « d »

```
liste_entiers = [12, 1543, 1765463]
for element in liste_entiers:
    print('| {:8d} |'.format(element)) # alignement sur 8 positions
```

```
|       12 |
|     1543 |
|  1765463 |
```

- Limiter les chiffres après la virgule dans les nombres décimaux

```
pi = 3.141592653589793
e = 2.7182818
dix = 10.0
liste_decimaux = [pi, e, dix]
for element in liste_decimaux:
    print('| {:.4f} |'.format(element)) # 4 chiffres après la virgule
```

```
| 3.1416 |
| 2.7183 |
| 10.0000 |
```

Les modules

Les fonctions préinstallées

Le formatage des données avec contraintes :

- Limiter le nombre de caractères pour les nombres décimaux

```
pi = 3.141592653589793
e = 2.7182818
dix = 10.0
liste_decimaux = [pi, e, dix]
for element in liste_decimaux:
    print('| {:6.2f} |'.format(element)) # 6 caractères en tout dont 2 après la virgule
```

```
|   3.14 |
|   2.72 |
|  10.00 |
```

- Plus de possibilités:
  - https://pyformat.info/
  - https://docs.python.org/fr/3.5/library/string.html#formatstrings

Les modules

Le mot clé « import »

- Pour utiliser une fonction prédéfinie non préinstallée dans la bibliothèque standard , il faudra importer le module qui la contient.
- Puisqu'un module contient une multitude de fonctions, il est souvent utile d'importer uniquement les fonctions à utiliser.

```
from math import sin # Importe la fonction sinus
from math import cos, sin, tan, pi # Importe respectivement les fonction cosinus, sinus, tangente et la valeur pi (3.14).
from math import * # Importe toutes les fonctions associées à math (équivalent à import math)
```

- Lorsque le nom du module est long, il peut être utile d'importer le module en utilisant un **alias** avec le mot clé « **as** » afin de lui attribuer un nom temporaire.

```
import matplotlib as plt
import random as rd
```

Les modules

Le module « math »

- C'est un module qui permet d'avoir accès aux fonctions mathématiques.
- Parmi les fonctions offertes : le cosinus (cos), le sinus (sin), la racine carrée (sqrt), le nombre **π** (pi) …

```
from math import * # importation du module
x = cos(pi)        # cosinus d'un angle en radian
y = sqrt(25)       # racine carrée d'un nombre
```

```
from math import *  # importation du module
x = floor(12.123)   # partie entière d'un nombre
print(x)
```

```
# Importer le module math
import math
x = math.pow(9, 3) # calcul de 9 puissance 3
print(x)
```

Sans le mot clé « from » et l'alias, il est obligatoire de mettre le nom du module avant l'appel de la fonction.

Les modules

Le module « math »

ATTENTION !!!

- La fonction round() ne fait pas partie du module math, elle s'applique directement sur un nombre de type float.
- **round(nombre, chiffres)** : permet de retourner un nombre flottant arrondi à « chiffres » valeurs après la virgule.
- **round(nombre) :** permet de retourner l'entier le plus proche

```
x = round(5.86743, 2)
print(x)
print(round(5.86743, 0))
print(round(5.86743))
```

```
5.87
6.0
6
```

Les modules

Le module « math »

- Pour afficher l'ensemble des fonctions contenue dans le module « math »

```
print(dir(math))
```

- Pour avoir l'aide sur une fonction particulière dans un le module « math » :

```
# Affiche l'aide à propos de la fonction prédéfinie gcd()
print(help(math.gcd()))
```

Les modules

Le module « random »

- Ce module implémente des générateurs de nombres pseudo-aléatoires pour différentes distributions.
    - Pour les entiers, il existe une sélection uniforme à partir d'une plage.
    - Pour les séquences (listes), il existe une sélection uniforme d'un élément aléatoire.
    - Pour l'ensemble des réels, il y a des fonctions pour calculer des distributions uniformes, normales (gaussiennes), log-normales, exponentielles négatives, …

Les modules

Le module « random »

Générateur d'un nombre aléatoire : seed()

- La fonction seed() permet de générer la base à partir de laquelle les nombres aléatoires seront générés.
- Utile pour reproduire une séquence de nombres aléatoires si la fonction est utilisée avec la même valeur

```
# Importer le module random
import random
random.seed(2) # initialiser le générateur de nombres aléatoires
alea = random.random()  # génération d'un nombre aléatoire dans l'intervalle [0.0, 1.0[
print(alea)
```

Les modules

Le module « random »

- **random()** : La fonction la plus basique. Elle génère un nombre aléatoire entre 0 et 1 , 1 exclu.
- **randrange()** and **randint()** : fonctions générant des nombres aléatoires de types entiers

```
# Importer le module random
import random
random.seed(3) # initialiser le générateur de nombres aléatoires
alea1 = random.randint(0, 9)  # nombre aléatoire dans l'intervalle [0, 9]
alea2 = random.randrange(2, 20, 2) # nombre aléatoire de 2 à 20 avec un pas égal à 2
```

- **choice()**: choisit une valeur aléatoire parmi une séquence (liste) de valeurs.

```
# Importer le module random
import random
alea = random.choice(['a', 'b', 'c']) # choix aléatoire d'un item de la liste ['a',
'b', 'c']
```

Les modules

Les modules « time » et « datetime »

- **time.time():** Pour avoir le temps courant en seconde, avec une précision à la microseconde.

```
# Importer le module time
import time
print(time.time())
```

- **time.sleep(s)** : Pour suspendre le process pendant **s** secondes.

```
# Importer le module time
import time
print("Bonjour")
time.sleep(2.3) # Attente de 2.3 secondes avant l'exécution de la prochaine action
print("tout le monde")
```

- **datetime.date.today():** Pour afficher la date du jour

```
# Importer le module datetime
import datetime as dt
print(dt.date.today())
```

Les modules

Les modules « time » et « datetime »

- **datetime.datetime.today() :** Pour avoir la date et l'heure courantes

```
# Importer le module datetime
import datetime as dt
print(dt.datetime.today())
```

- **datetime.date(annee, mois, jour)** : Construit une date à partir de annee, mois et jour.

```
# Importer le module datetime
import datetime as dt
print(dt.date(2024, 10, 25))
```

- **datetime.datetime(annee, mois, jour, heure, minutes, secondes)**

```
# Importer le module datetime
import datetime as dt
print(dt.datetime(2024, 10, 25, 13, 50, 00))
```

Les modules

Le module graphique « turtle »

- Le module **turtle** de Python permet de commander une tortue qui va tracer des segments.

- Module: `import turtle`

- Fonctionnalités :
  - Commander le paramétrage du crayon
  - Déplacer la tortue
  - Déplacer la tortue à un point précis
  - Écrire du texte à un endroit précis

Les modules

Le module graphique « turtle »

- **turtle.down() :** abaisse le stylo
- **turtle.up()** : relève le stylo
- **turtle.pensize(width)** : change l'épaisseur du trait
- **turtle.pencolor(color)** : change la couleur (*"red"*, *"green"*, *"blue"*… ou un triplet de paramètres (r, g, b))
- **turtle.forward(length)** : avance de length pas
- **turtle.backward(length)** : recule de length pas
- **turtle.right(angle)** , **turtle.left(angle)** : tourne vers la droite / gauche d'un angle donné (en degrés)

Les modules

Le module graphique « turtle »

- **turtle.goto(x,y) :** déplace la tortue jusqu'au point (x, y) sachant que :
  - Le point $(0, 0)$ est au centre de l'écran.
  - La fenêtre par défaut est $950 = 2 \times 475$ pixels de large et $800 = 2 \times 400$ pixels de haut.
- **turtle.setup(width, height)** : définit une nouvelle taille de la fenêtre
- **turtle.clear()** : efface tout ce qui a été tracé ou écrit dans la fenêtre.
- **turtle.write(str) :** écrit la chaîne de caractères donnée à la position courante

Les modules

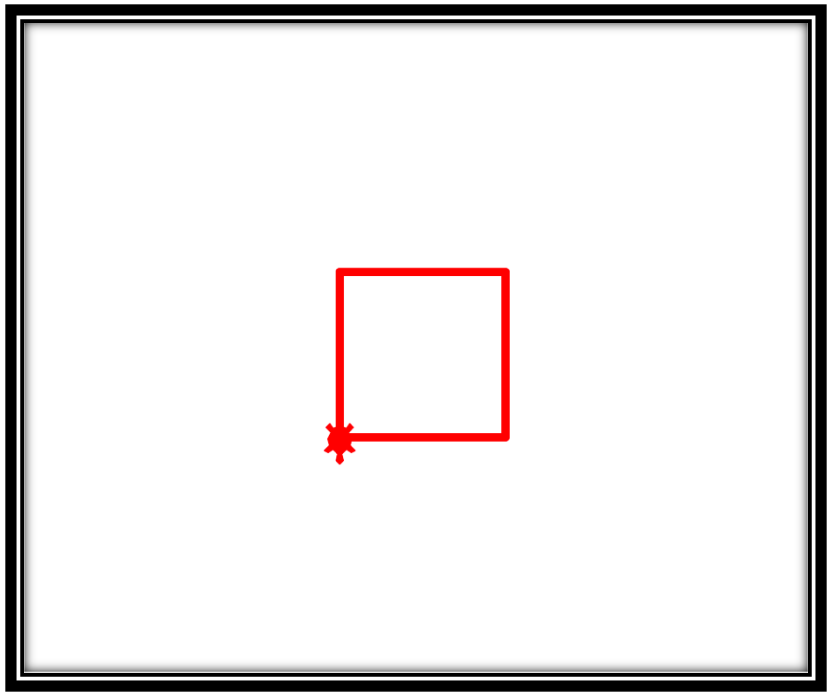
Le module graphique « turtle »

Exemple :

```
import turtle

tim = turtle.Turtle()
tim.color("red")
tim.pensize(5)
tim.shape("turtle")

tim.forward(100)
tim.left(90)
tim.forward(100)
tim.left(90)
tim.forward(100)
tim.left(90)
tim.forward(100)
```



03

Activité

03

Activité

Exercice 1 :

Soit le fichier « nombres.txt » contenant les valeurs réelles suivantes:

```
14.896
7894.6
123.278
```

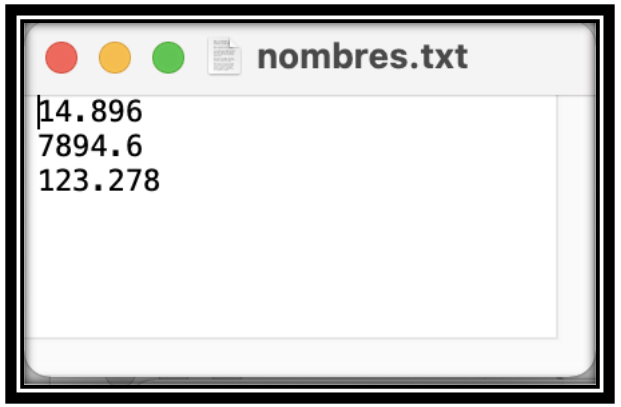
- Écrire un programme Python permettant de:
  - Lire le fichier « nombres.txt » ligne par ligne
  - Prend la partie entière de chaque nombre
  - La copie dans un nouveau fichier « nombresInt.txt »

Activité

Exercice 1 : Solution

```python
import math

with open("nombres.txt", "r") as f1, open("nombresInt.txt", "w") as f2:
    line = f1.readline()
    while line != "":
        # Convertir la ligne en un float
        line_float = float(line)
        # Prendre la partie entière
        floor_part = math.floor(line_float)
        # Convertir le nombre entier en chaîne de caractères
        str_floor_part = str(floor_part)
        # Écrire le nombre entier dans f2
        f2.write(str_floor_part + "\n")
        # Lire la ligne suivante
        line = f1.readline()
```



Activité

Exercice 2 :

Analyser puis modifier le code Python qui suit afin d'obtenir ce rectangle vert au contour bleu.

```
# Programme pour dessiner un carré avec Python Turtle

import turtle

t = turtle.Turtle()
t.forward(100)  # Avancer la tortue de 100 unités
t.left(90)  # Tourner la tortue de 90 degrés
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
```

Activité

Exercice 2 : Solution

```
# Programme pour dessiner un carré avec Python Turtle
import turtle
import time

t = turtle.Turtle()
t.color("blue")
t.pensize(5)
t.fillcolor("green")
t.begin_fill()
t.forward(120) # Avancer la tortue de 100 unités
t.left(90) # Tourner la tortue de 90 degrés
t.forward(70)
t.left(90)
t.forward(120)
t.left(90)
t.forward(70)
t.left(90)
t.end_fill()

time.sleep(20)
```